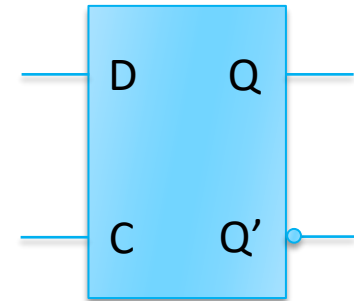


Elementi di memoria

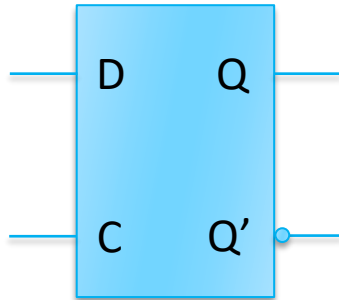
Latch D

- ▶ Il latch D è *trasparente* quando $C = 1$
 - ▶ L'uscita segue le variazioni dell'ingresso
- ▶ Il latch D è in *memoria* quando $C = 0$
 - ▶ Le variazioni su D non hanno influenza sull'uscita
- ▶ Si comporta come una porta
 - ▶ Quando $C = 1$ la porta è aperta
 - ▶ Quando $C = 0$ la porta si chiude e l'uscita **mantiene l'ultimo valore che c'era su D prima che avvenisse la transizione su C**
 - ▶ **Diverso da una porta logica**, che non ricorda

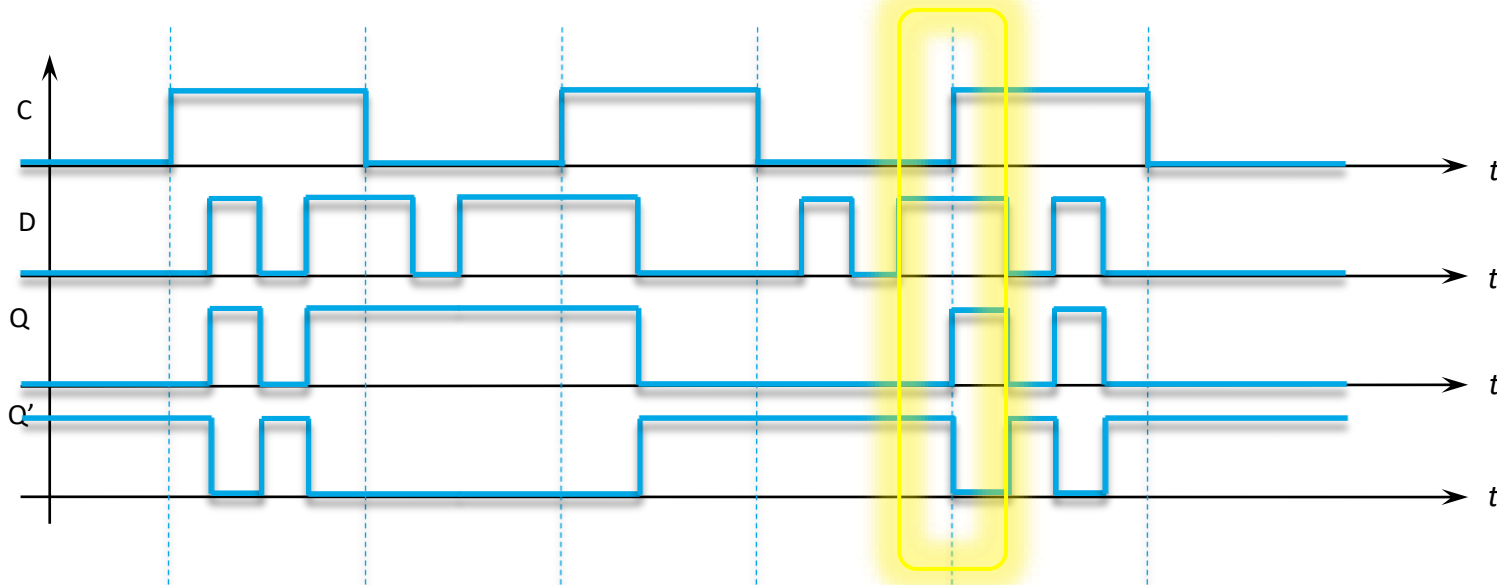


C	D	Q	Q'
0	-	Q_{-1}	Q'_{-1}
1	0	0	1
1	1	1	0

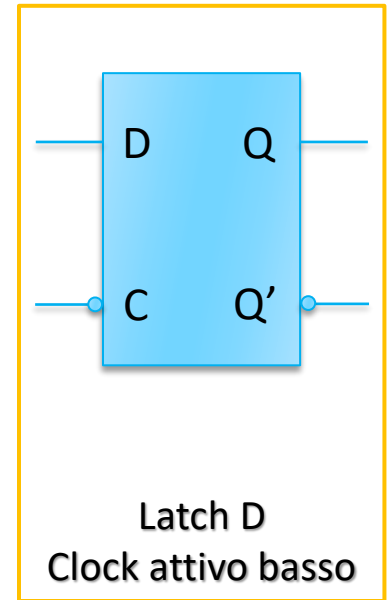
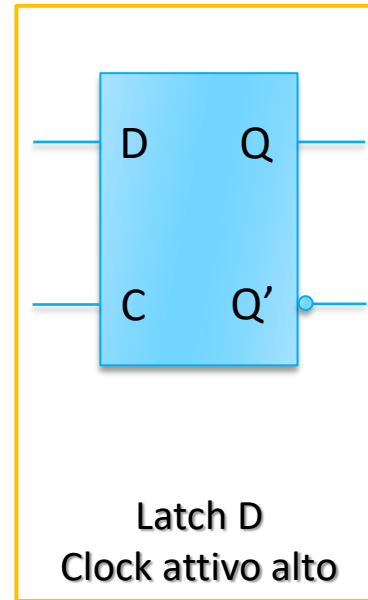
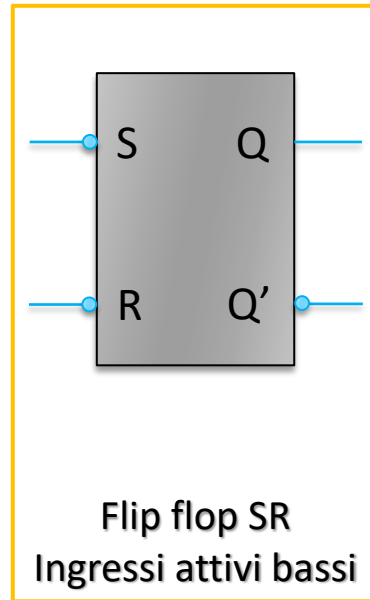
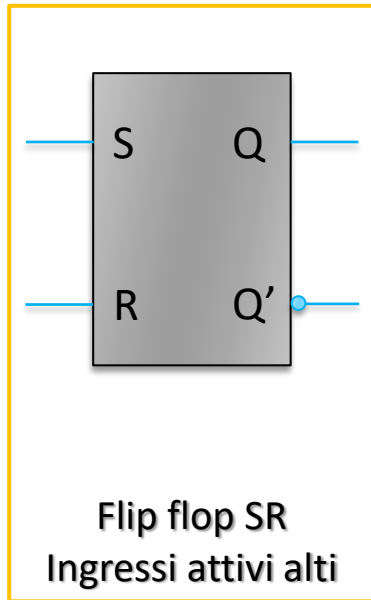
Diagramma temporale



- ▶ **L'uscita Q segue D quando C = 1**
 - ▶ Memorizza quando C diventa 0
 - ▶ Q può fare una transizione quando C diventa 1, e il valore su D è diverso da quello memorizzato

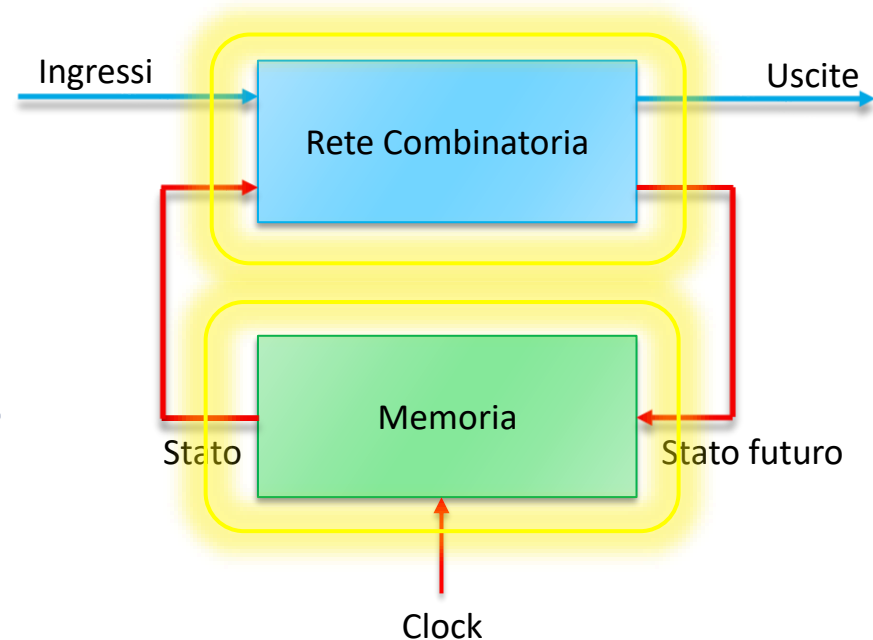


Simboli grafici



Rete sequenziale

- ▶ **Latch D usato per costruire la parte di memoria di un circuito sequenziale**
 - ▶ Il segnale C, detto segnale di *clock*, definisce quando lo stato può cambiare

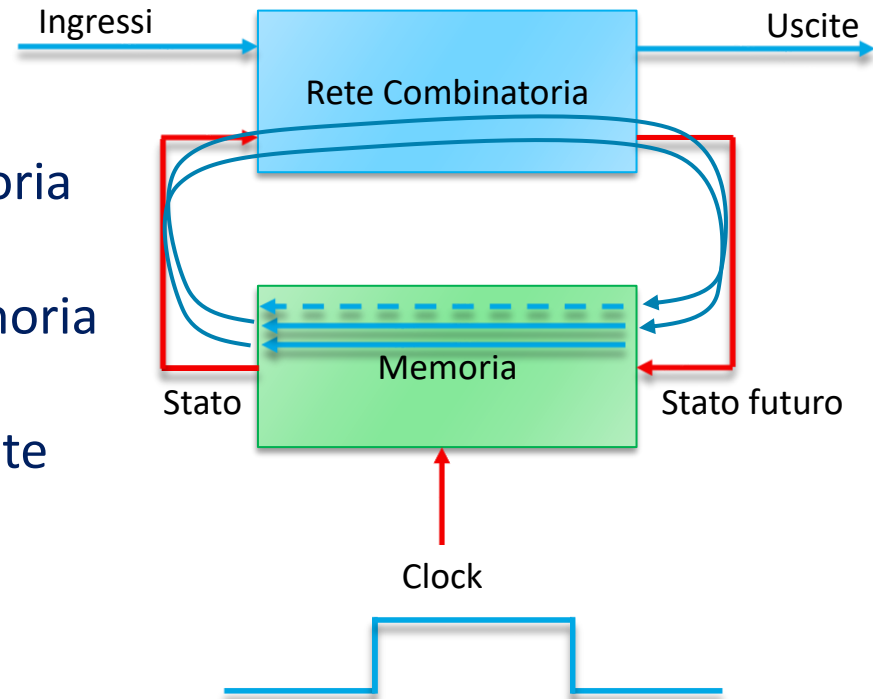


- ▶ **Il circuito passa attraverso due fasi**
 - ▶ 1. La rete combinatoria calcola le nuove uscite e il nuovo stato futuro mentre il clock è basso
 - ▶ 2. Il clock si attiva, e lo stato futuro diventa stato presente
 - ▶ Il ciclo si ripete infinitamente

Stabilità

► Il latch è trasparente mentre il clock è attivo

- Supponiamo che la rete combinatoria sia molto veloce
- Il nuovo stato si presenta alla memoria *prima* che il clock si sia disattivato
- Il nuovo stato diventa stato presente
- Si calcola un nuovo stato futuro



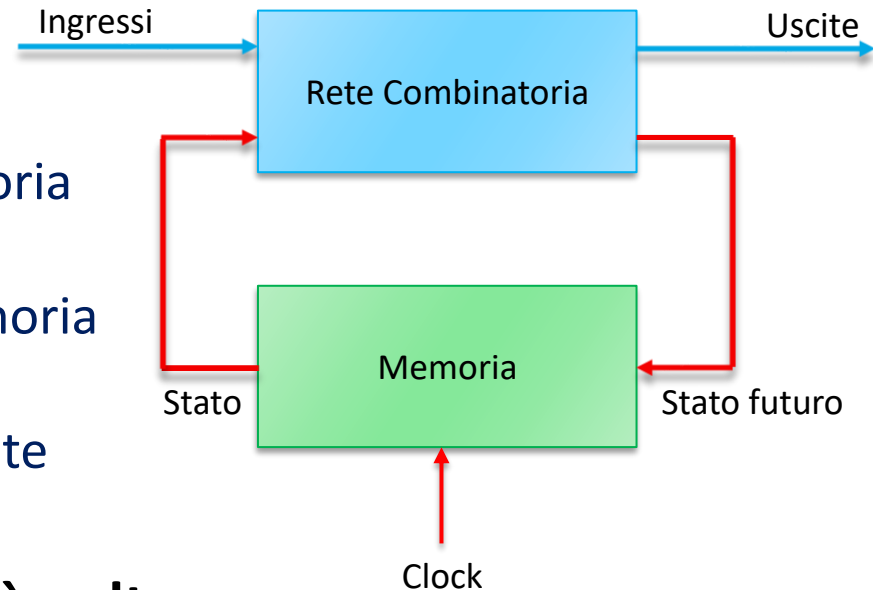
Stabilità

► Il latch è trasparente mentre il clock è attivo

- Supponiamo che la rete combinatoria sia molto veloce
- Il nuovo stato si presenta alla memoria *prima* che il clock si sia disattivato
- Il nuovo stato diventa stato presente
- Si calcola un nuovo stato futuro

► Il circuito può cambiare stato più volte durante una fase attiva del clock

- Rende la rete poco affidabile, instabile, non abbiamo risolto nulla!
- Il numero di cambi di stato dipende dalla velocità del circuito
- Le variazioni di fabbricazione rendono difficile realizzare reti tutte con lo stesso comportamento



Esempio di circuito instabile

- ▶ **Riportiamo l'uscita negata sull'ingresso**

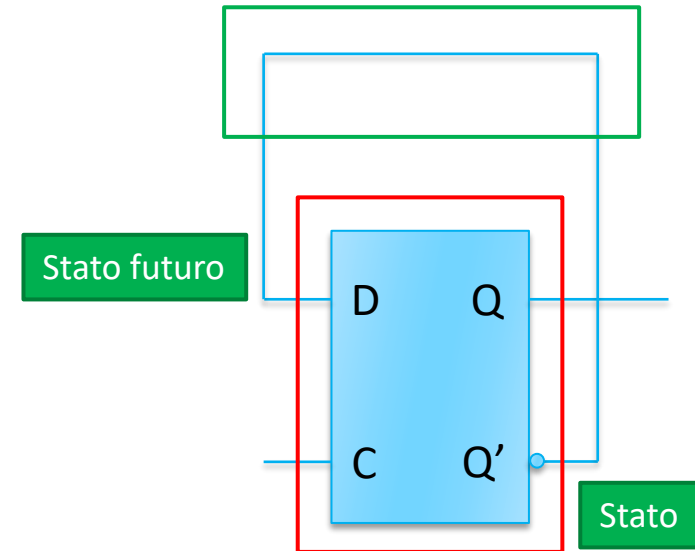
- ▶ La rete combinatoria è semplicemente un filo
- ▶ La rete di memoria è formata dal latch

- ▶ **Applichiamo a C un clock**

- ▶ Quando $C = 1$ il circuito comincia ad oscillare
- ▶ Quando $C = 0$ il circuito si ferma sull'ultimo valore
- ▶ L'oscillazione dipende dai ritardi attraverso il flip flop

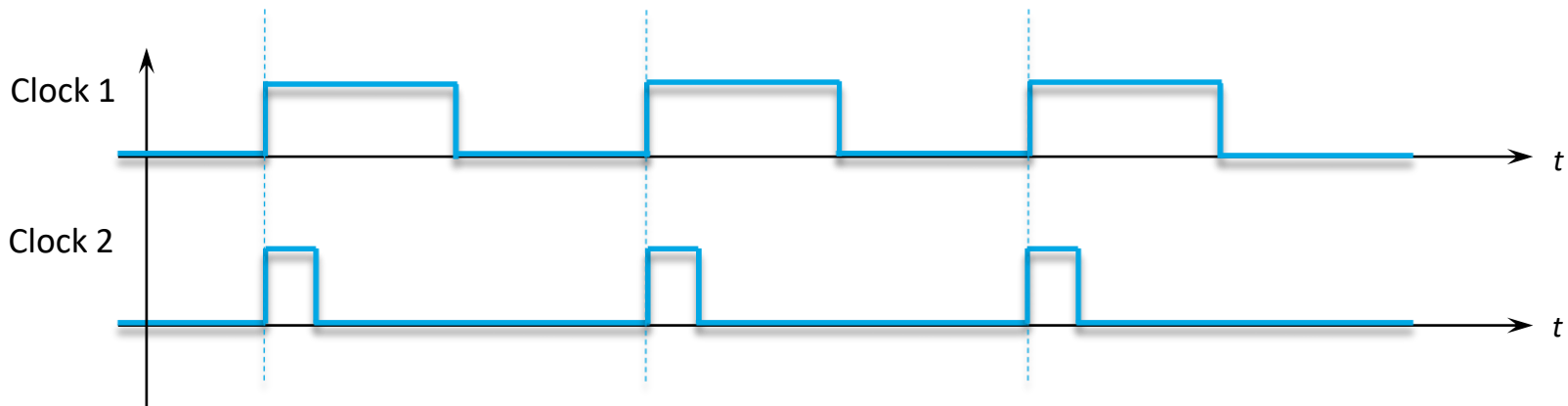
- ▶ **Cosa volevamo?**

- ▶ L'intenzione era quella di far cambiare Q solamente *una volta* per ogni impulso attivo del clock



Soluzioni

- ▶ **Il latch D è sensibile al *livello* del clock**
 - ▶ E' attivo per tutto il tempo in cui il clock è a 1
- ▶ **Occorre avere una fase attiva molto breve**
 - ▶ Il clock sta sempre a 0 tranne per un breve periodo
 - ▶ Il periodo attivo deve essere più breve del tempo di propagazione minimo attraverso la rete combinatoria



Realizzazione e problemi

► Il clock asimmetrico crea problemi

- Quando la logica è veloce (per esempio un filo!), la fase attiva deve essere estremamente stretta
- Occorrono transistori grossi per fare transizioni il più possibile verticali
- Tenete presente che ci sono migliaia di elementi di memoria in un circuito sequenziale (alta capacità sulla linea del clock)

► La fase stretta aumenta il contenuto armonico

- Più problemi di compatibilità elettromagnetica
- Spreco di energia

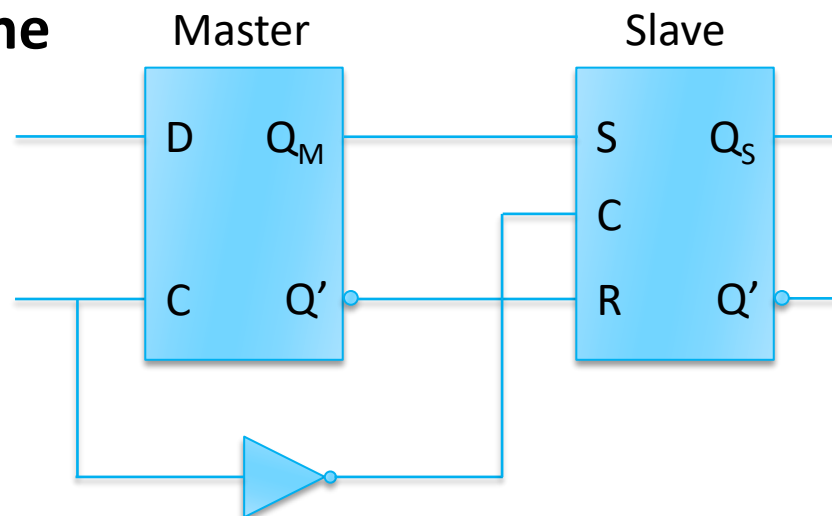
► Il problema è che il latch è trasparente

- Ci vorrebbe un elemento di memoria che non sia mai trasparente
- Tranne che per un istante in cui si trasferisce il dato dall'ingresso all'uscita

Flip flop di tipo edge triggered

► Due latch con clock in opposizione

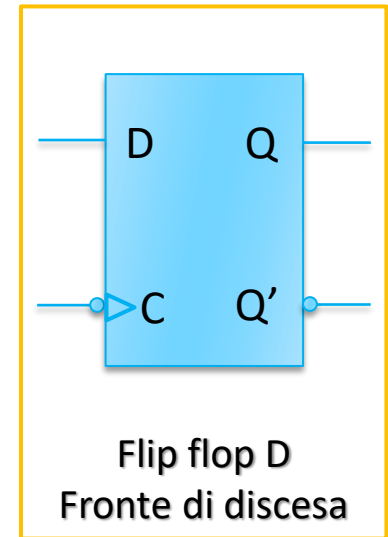
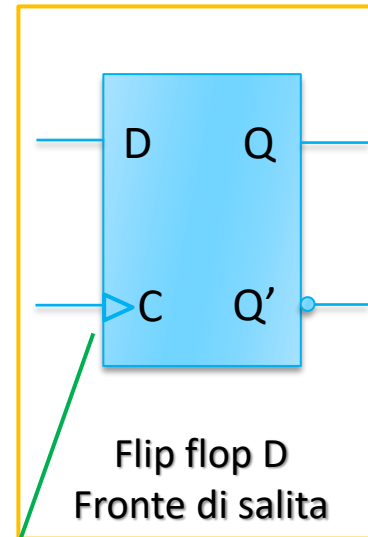
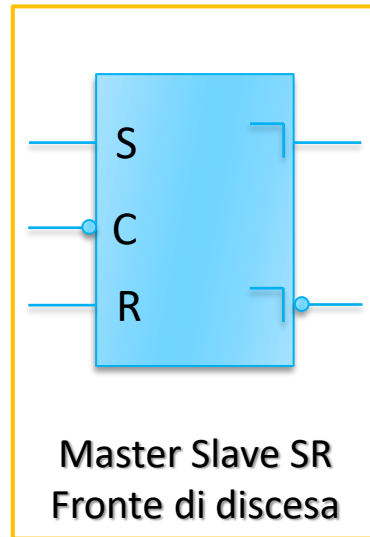
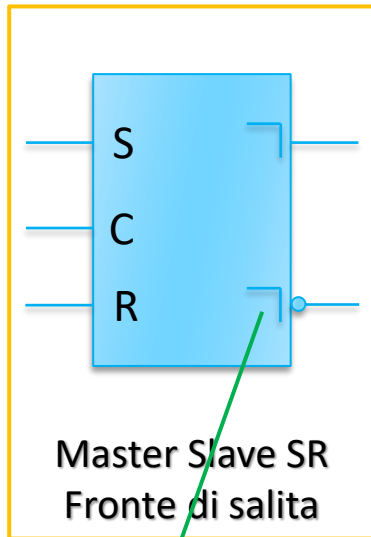
- Il primo, detto *master*, aperto quando il clock è alto
- Il secondo, detto *slave*, aperto quando il clock è basso
- Non c'è mai un cammino diretto tra ingresso e uscita



► Funzionamento

- Quando $C = 1$ il primo flip flop è aperto e Q_M segue le variazioni di D , ma l'uscita sta ferma perché lo slave non è abilitato
- Quando C esegue la transizione da 1 a 0 il valore di Q_M (che è anche quello di D) viene trasferito nel secondo flip flop e sull'uscita
- Mentre $C = 0$ tutto sta fermo perché il primo non è sensibile, e il secondo ha gli ingressi fissi

Simboli grafici



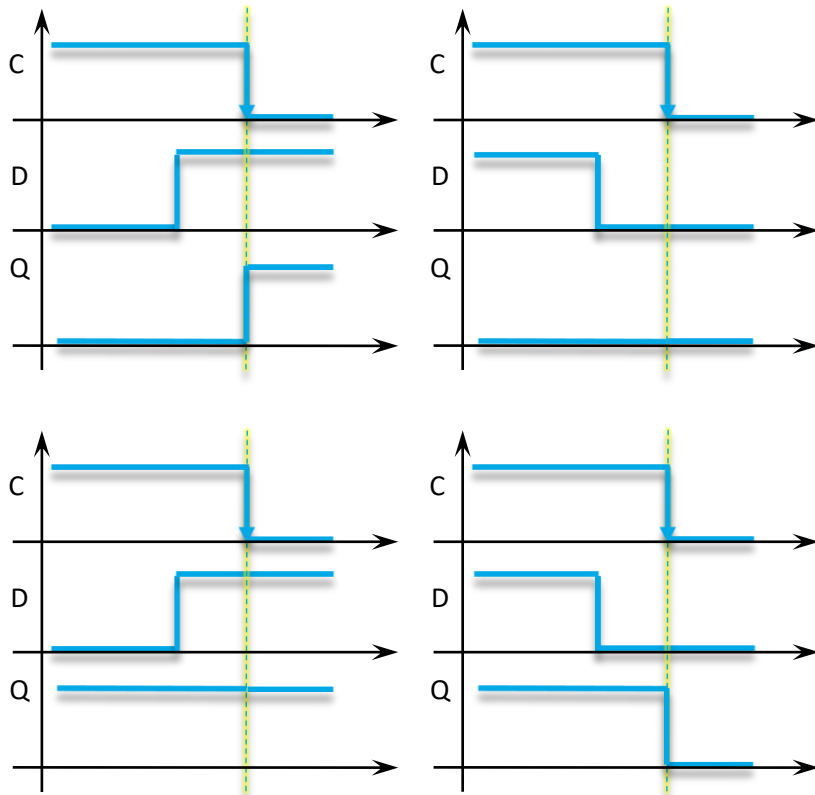
Uscite aggiornate
al fronte

Edge triggered

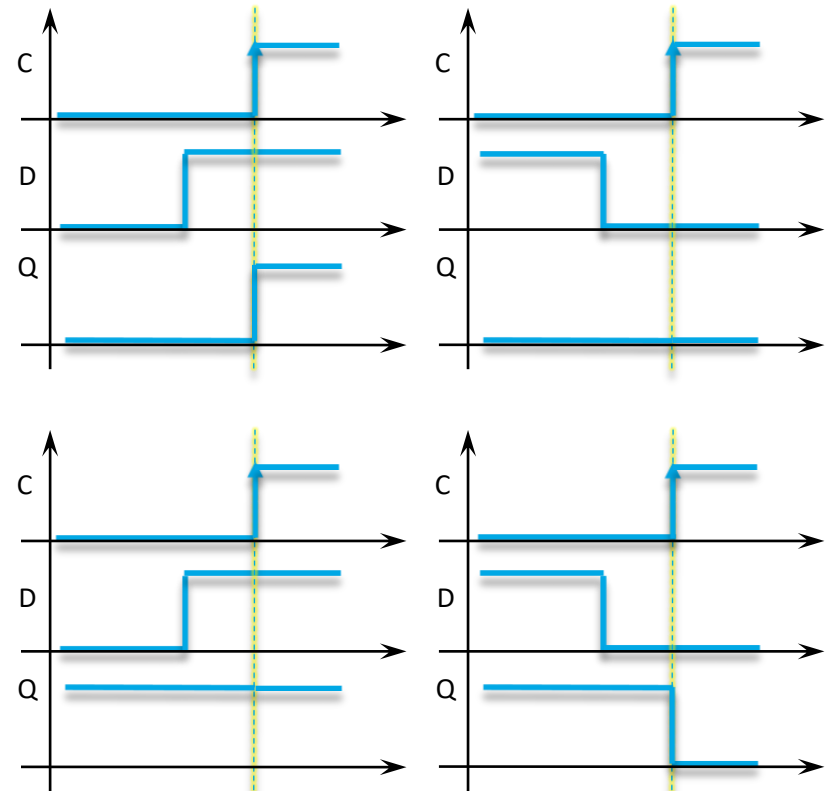
Negative e positive edge triggered

► Comportamento al fronte

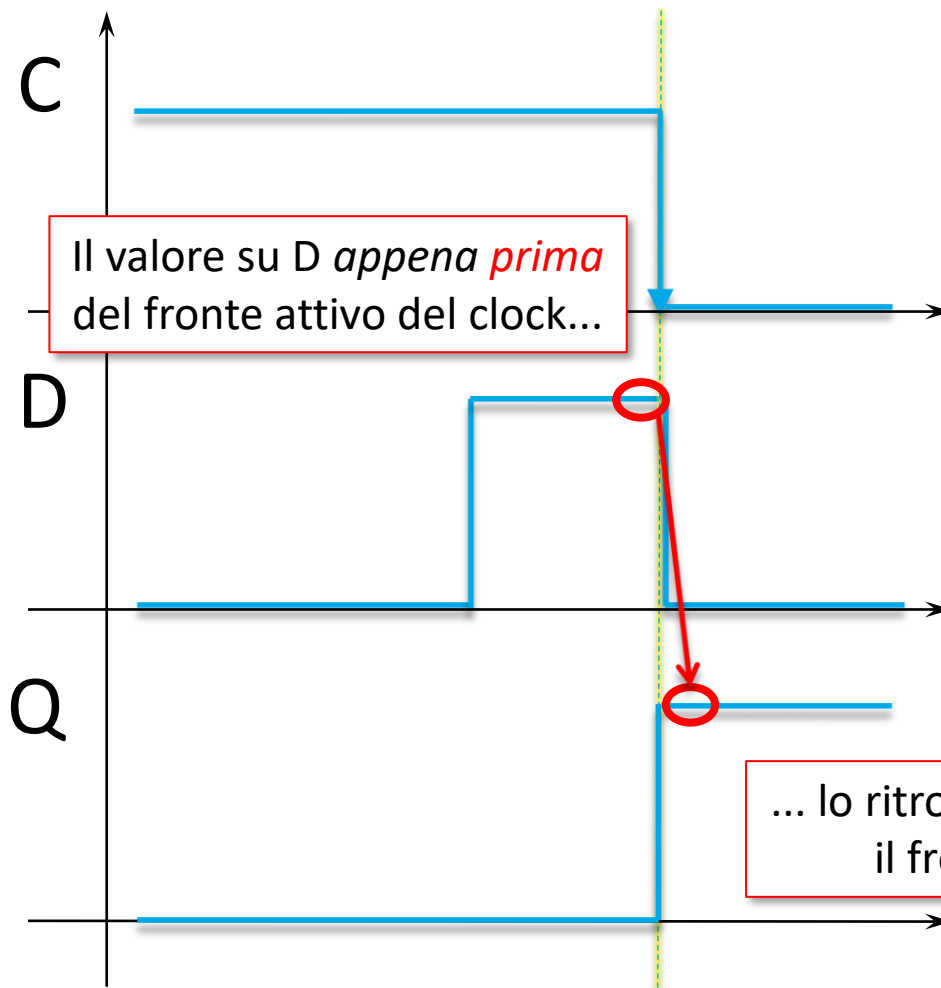
Negative edge triggered



Positive edge triggered



Più in dettaglio (importante)



► Il fronte separa un ciclo di clock dal successivo

- Lo stato futuro su D viene trasferito come stato presente su Q sul fronte
- D può fare quello che vuole in tutto il resto del tempo, **anche avere glitch**
- L'importante che abbia il valore corretto nel momento in cui arriva il fronte

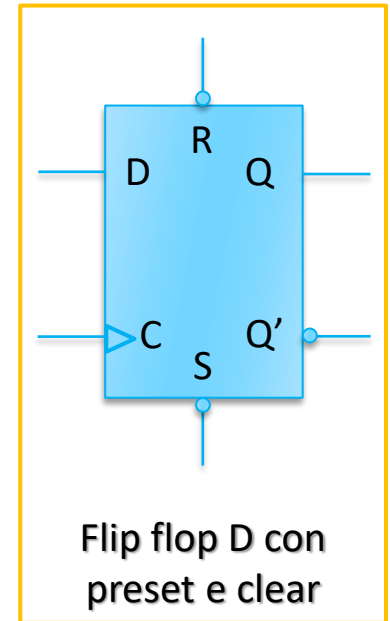
Preset e clear

► E' necessario dare un valore iniziale al flip flop

- Altrimenti il valore iniziale è ignoto
- Lo si può fare al primo fronte attivo del clock
- Oppure si utilizzano spesso segnali asincroni di inizializzazione
- Asincroni significa che hanno effetto immediatamente, anche non al fronte del clock

► Preset e clear

- Indicati con S ed R
- Spesso attivi bassi
- Ininfluenti quando a 1
- Se $S = 0$, Q si porta immediatamente a 1
- Se $R = 0$, Q si porta immediatamente a 0



Flip flop D con
preset e clear

- ▶ **Possibili molti tipi di elementi di memoria diversi**
 - ▶ Ne vedremo anche altri
 - ▶ Ognuno è un diverso compromesso tra complessità e caratteristiche
- ▶ **Vogliamo evitare le instabilità**
 - ▶ Dividiamo la parte combinatoria (calcolo) da quella sequenziale (stato)
 - ▶ Selezioniamo gli istanti in cui è possibile far variare lo stato attraverso un segnale di clock
 - ▶ Dobbiamo comunque però evitare degli anelli di feedback
 - ▶ Usiamo flip flop di tipo edge triggered
- ▶ **Disaccoppiamento tra stato futuro e stato presente**
 - ▶ Lo stato futuro deve essere a regime entro la fine del ciclo di clock
 - ▶ Nel frattempo può anche avere dei glitch, ma questi non si propagano attraverso il flip flop edge triggered
 - ▶ **I glitch non influiscono sulla rete combinatoria**